Professores
Celso Giusti
Daniel Filho
Felipe Gregate
Marlon Rodrigues

Arquitetura MVC: Organizando o Caos no Desenvolvimento de APIs

SENAI

Objetivos

Nesta aula, buscaremos atingir as seguintes capacidades e conhecimentos:

•**Compreender** o conceito e o funcionamento do padrão arquitetural MVC (Model-View-Controller).
•**Identificar** as responsabilidades individuais de cada camada: Modelo, Visão e Controlador.
•**Diferenciar** a arquitetura MVC de modelos monolíticos e outras abordagens de camadas.
•**Analisar** os benefícios da Separação de Preocupações (SoC) na escalabilidade de sistemas.

O Problema da Arquitetura Monolítica

O Caos do "Tudo no Mesmo Lugar“

•Em sistemas simples, é comum misturar lógica de negócio, acesso a dados e resposta HTTP em um único arquivo.

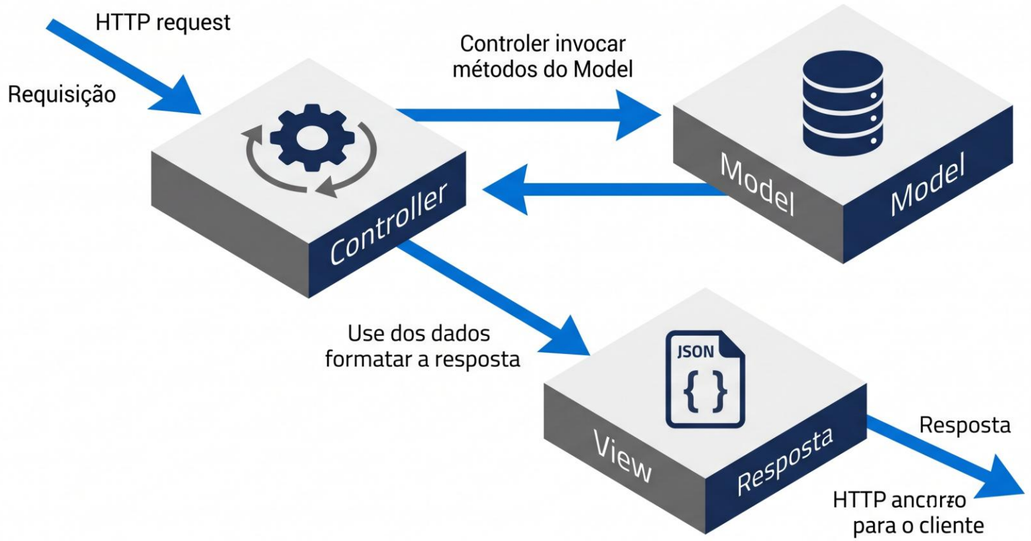
•**Consequência:** Dificuldade extrema de manutenção.

•Um pequeno ajuste (como adicionar um campo) pode levar horas e gerar bugs inesperados em outras partes do sistema.

O Conceito MVC

A Planta Baixa do Software

- O MVC divide a aplicação em **três componentes principais**.
- Funciona como uma planta arquitetônica: define onde cada peça da lógica deve residir.
- Baseia-se no princípio de **Separação de Preocupações (SoC)**.



O Model (Modelo)

O Especialista em Dados

- **Responsabilidade:** Gerenciar os dados e as regras de negócio.
- Comunica-se diretamente com o banco de dados.
- Valida as informações (Ex: formato de e-mail ou estoque disponível).

A View (Visão)

A Apresentação dos Dados

- **Responsabilidade:** Formatar os dados para o usuário final.
- Em APIs REST, a "cara" da aplicação é geralmente um arquivo **JSON**.
- Não deve conter lógica de negócio, apenas a exibição do que foi processado.

O Controller (Controlador)

O Maestro da Orquestra

- **Responsabilidade:** Intermediar a interação do usuário.
- Recebe a requisição HTTP (Ex: GET /produtos).
- Interpreta o pedido e chama o método correto no **Model**.

Comparativo: Monolítico vs. MVC

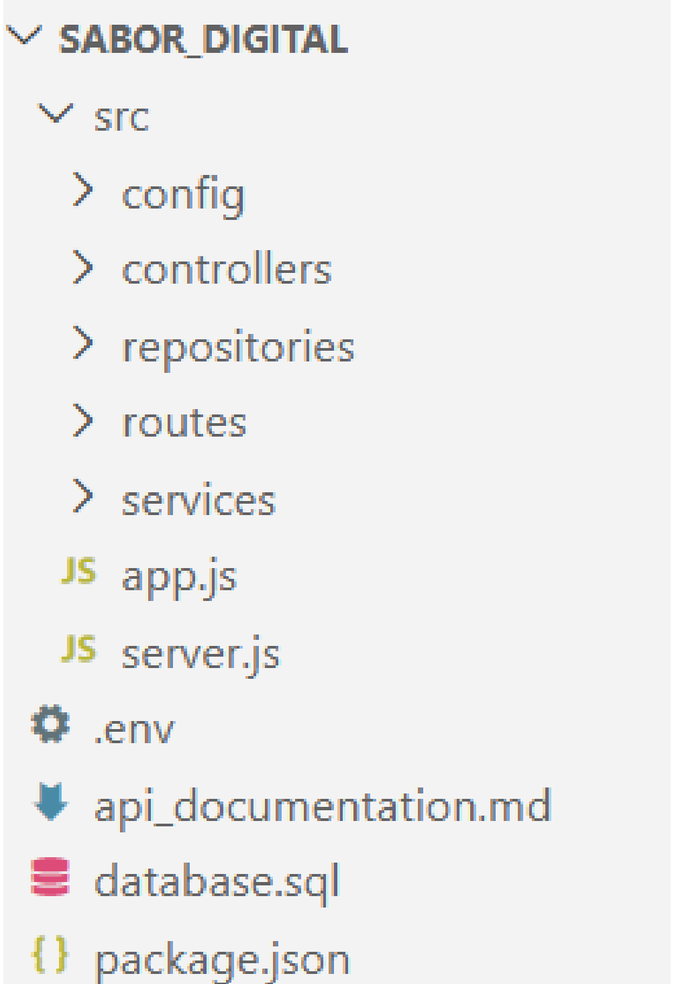
Por que mudar?

Característica	Arquitetura Monolítica	Arquitetura MVC
Organização	Tudo misturado em um arquivo	Divisão clara de funções
Manutenção	Alto risco de quebrar o sistema	Alterações localizadas e seguras
Testabilidade	Difícil de testar isoladamente	Fácil testar cada camada
Equipe	Desenvolvedores "se atropelam"	Trabalho paralelo facilitado

MVC e Camada de Serviço

Indo Além: Service Layer e Repository

- **MVC Básico:** O Controller fala direto com o Model.
- **Arquitetura em Camadas (Service):** Introduz o "Chef Executivo".
- O **Service** lida com regras complexas que envolvem múltiplos modelos.
- O **Repository** abstrai o banco de dados (permite trocar MySQL por MongoDB sem sofrer)



RESUMO DA AULA

✅ *MVC* = Padrão que divide a aplicação em Model, View e Controller para organizar o código.

✅ *Model* = Camada especialista em dados e regras de negócio (o "Cérebro").

✅ *View* = Representação dos dados para o cliente, geralmente JSON em APIs.

✅ *Controller* = Intermediário que recebe requisições e coordena Model e View (o "Maestro").

✅ *Benefício Central* = Separação de Preocupações, facilitando manutenção e testes.

REFERÊNCIAS

GIUSTI, Celso Rodrigo. **Técnicas de Codificação e Design para Sistemas Back-End**. São Paulo: SENAI-SP, [s.d.].

SENAI-SP. **Programação Back-End.** São Paulo: Editora SENAI, 2025.

SENAI-SP. **Plano de Curso: Técnico em Desenvolvimento de Sistemas**. São Paulo: Departamento Regional de São Paulo, 2023.

Parte do conteúdo dessa apresentação foi gerada por IA com o propósito de deixar a apresentação mais dinâmica.

Escola SENAI "Italo Bologna"

Av. Goiás, 139 – Itu/SP

Telefone
(11) 2396-1999

Instagram
@senai.itu

Facebook
/senai.itu

Site
https://sp.senai.br/unidade/itu/